

### 13.3 节练习

练习 13.29: 解释 `swap(HasPtr&, HasPtr&)` 中对 `swap` 的调用不会导致递归循环。

练习 13.30: 为你的类值版本的 `HasPtr` 编写 `swap` 函数, 并测试它。为你的 `swap` 函数添加一个打印语句, 指出函数什么时候执行。

练习 13.31: 为你的 `HasPtr` 类定义一个 `<` 运算符, 并定义一个 `HasPtr` 的 `vector`。为这个 `vector` 添加一些元素, 并对它执行 `sort`。注意何时会调用 `swap`。

练习 13.32: 类指针的 `HasPtr` 版本会从 `swap` 函数受益吗? 如果会, 得到了什么益处? 如果不是, 为什么?

## 13.4 拷贝控制示例

虽然通常来说分配资源的类更需要拷贝控制, 但资源管理并不是一个类需要定义自己的拷贝控制成员的唯一原因。一些类也需要拷贝控制成员的帮助来进行簿记工作或其他操作。

作为类需要拷贝控制来进行簿记操作的例子, 我们将概述两个类的设计, 这两个类可能用于邮件处理应用中。两个类命名为 `Message` 和 `Folder`, 分别表示电子邮件 (或者其他类型的) 消息和消息目录。每个 `Message` 对象可以出现在多个 `Folder` 中。但是, 任意给定的 `Message` 的内容只有一个副本。这样, 如果一条 `Message` 的内容被改变, 则我们从它所在的任何 `Folder` 来浏览此 `Message` 时, 都会看到改变后的内容。

为了记录 `Message` 位于哪些 `Folder` 中, 每个 `Message` 都会保存一个它所在 `Folder` 的指针的 `set`, 同样的, 每个 `Folder` 都保存一个它包含的 `Message` 的指针的 `set`。图 13.1 说明了这种设计思路。

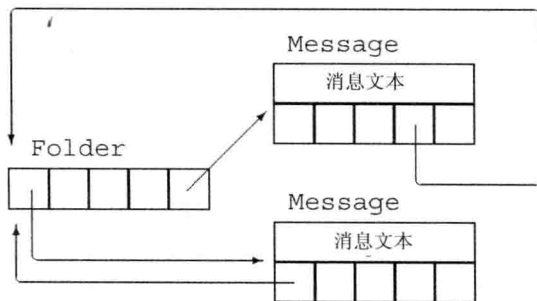


图 13.1: `Message` 和 `Folder` 类设计

我们的 `Message` 类会提供 `save` 和 `remove` 操作, 来向一个给定 `Folder` 添加一条 `Message` 或是从中删除一条 `Message`。为了创建一个新的 `Message`, 我们会指明消息内容, 但不会指出 `Folder`。为了将一条 `Message` 放到一个特定 `Folder` 中, 我们必须调用 `save`。

当我们拷贝一个 `Message` 时, 副本和原对象将是不同的 `Message` 对象, 但两个 `Message` 都出现在相同的 `Folder` 中。因此, 拷贝 `Message` 的操作包括消息内容和 `Folder` 指针 `set` 的拷贝。而且, 我们必须在每个包含此消息的 `Folder` 中都添加一个指向新创建的 `Message` 的指针。

当我们销毁一个 `Message` 时, 它将不复存在。因此, 我们必须从包含此消息的所有

Folder 中删除指向此 Message 的指针。

当我们将一个 Message 对象赋予另一个 Message 对象时, 左侧 Message 的内容会被右侧 Message 的内容所替代。我们还必须更新 Folder 集合, 从原来包含左侧 Message 的 Folder 中将它删除, 并将它添加到包含右侧 Message 的 Folder 中。

观察这些操作, 我们可以看到, 析构函数和拷贝赋值运算符都必须从包含一条 Message 的所有 Folder 中删除它。类似的, 拷贝构造函数和拷贝赋值运算符都要将一个 Message 添加到给定的一组 Folder 中。我们将定义两个 private 的工具函数来完成这些工作。



拷贝赋值运算符通常执行拷贝构造函数和析构函数中也要做的工作。这种情况下, 公共的工作应该放在 private 的工具函数中完成。

Folder 类也需要类似的拷贝控制成员, 来添加或删除它保存的 Message。

521

我们将 Folder 类的设计和实现留作练习。但是, 我们将假定 Folder 类包含名为 addMsg 和 remMsg 的成员, 分别完成在给定 Folder 对象的消息集合中添加和删除 Message 的工作。

## Message 类

根据上述设计, 我们可以编写 Message 类, 如下所示:

```
class Message {
    friend class Folder;
public:
    // folders 被隐式初始化为空集合
    explicit Message(const std::string &str = ""):
        contents(str) { }
    // 拷贝控制成员, 用来管理指向本 Message 的指针
    Message(const Message&);           // 拷贝构造函数
    Message& operator=(const Message&); // 拷贝赋值运算符
    ~Message();                       // 析构函数
    // 从给定 Folder 集合中添加/删除本 Message
    void save(Folder&);
    void remove(Folder&);
private:
    std::string contents;           // 实际消息文本
    std::set<Folder*> folders;      // 包含本 Message 的 Folder
    // 拷贝构造函数、拷贝赋值运算符和析构函数所使用的工具函数
    // 将本 Message 添加到指向参数的 Folder 中
    void add_to_Folders(const Message&);
    // 从 folders 中的每个 Folder 中删除本 Message
    void remove_from_Folders();
};
```

这个类定义了两个数据成员: contents, 保存消息文本; folders, 保存指向本 Message 所在 Folder 的指针。接受一个 string 参数的构造函数将给定 string 拷贝给 contents, 并将 folders (隐式) 初始化为空集。由于此构造函数有一个默认参数, 因此它也被当作 Message 的默认构造函数 (参见 7.5.1 节, 第 260 页)。

## save 和 remove 成员

除拷贝控制成员外, Message 类只有两个公共成员: save, 将本 Message 存放在给定 Folder 中; remove, 删除本 Message:

```
void Message::save(Folder &f)
{
    folders.insert(&f); // 将给定 Folder 添加到我们的 Folder 列表中
    f.addMsg(this);     // 将本 Message 添加到 f 的 Message 集合中
}
522 void Message::remove(Folder &f)
{
    folders.erase(&f); // 将给定 Folder 从我们的 Folder 列表中删除
    f.remMsg(this);    // 将本 Message 从 f 的 Message 集合中删除
}
```

为了保存(或删除)一个 Message, 需要更新本 Message 的 folders 成员。当 save 一个 Message 时, 我们应保存一个指向给定 Folder 的指针; 当 remove 一个 Message 时, 我们要删除此指针。

这些操作还必须更新给定的 Folder。更新一个 Folder 的任务是由 Folder 类的 addMsg 和 remMsg 成员来完成的, 分别添加和删除给定 Message 的指针。

## Message 类的拷贝控制成员

当我们拷贝一个 Message 时, 得到的副本应该与原 Message 出现在相同的 Folder 中。因此, 我们必须遍历 Folder 指针的 set, 对每个指向原 Message 的 Folder 添加一个指向新 Message 的指针。拷贝构造函数和拷贝赋值运算符都需要做这个工作, 因此我们定义一个函数来完成这个公共操作:

```
// 将本 Message 添加到指向 m 的 Folder 中
void Message::add_to_Folders(const Message &m)
{
    for (auto f : m.folders) // 对每个包含 m 的 Folder
        f->addMsg(this);    // 向该 Folder 添加一个指向本 Message 的指针
}
```

此例中我们对 m.folders 中每个 Folder 调用 addMsg。函数 addMsg 会将本 Message 的指针添加到每个 Folder 中。

Message 的拷贝构造函数拷贝给定对象的数据成员:

```
Message::Message(const Message &m):
    contents(m.contents), folders(m.folders)
{
    add_to_Folders(m); // 将本消息添加到指向 m 的 Folder 中
}
```

并调用 add\_to\_Folders 将新创建的 Message 的指针添加到每个包含原 Message 的 Folder 中。

## Message 的析构函数

当一个 Message 被销毁时, 我们必须从指向此 Message 的 Folder 中删除它。拷贝赋值运算符也要执行此操作, 因此我们会定义一个公共函数来完成此工作:

```
// 从对应的 Folder 中删除本 Message
void Message::remove_from_Folders()
{
    for (auto f : folders)    // 对 folders 中每个指针
        f->remMsg(this);      // 从该 Folder 中删除本 Message
}
```

函数 `remove_from_Folders` 的实现类似 `add_to_Folders`，不同之处是它调用 `remMsg` 来删除当前 `Message` 而不是调用 `addMsg` 来添加 `Message`。 ◀ 523

有了 `remove_from_Folders` 函数，编写析构函数就很简单了：

```
Message::~Message()
{
    remove_from_Folders();
}
```

调用 `remove_from_Folders` 确保没有任何 `Folder` 保存正在销毁的 `Message` 的指针。编译器自动调用 `string` 的析构函数来释放 `contents`，并自动调用 `set` 的析构函数来清理集合成员使用的内存。

### Message 的拷贝赋值运算符

与大多数赋值运算符相同，我们的 `Message` 类的拷贝赋值运算符必须执行拷贝构造函数和析构函数的工作。与往常一样，最重要的是我们要组织好代码结构，使得即使左侧和右侧运算对象是同一个 `Message`，拷贝赋值运算符也能正确执行。

在本例中，我们先从左侧运算对象的 `folders` 中删除此 `Message` 的指针，然后再将指针添加到右侧运算对象的 `folders` 中，从而实现了自赋值的正确处理：

```
Message& Message::operator=(const Message &rhs)
{
    // 通过先删除指针再插入它们来处理自赋值情况
    remove_from_Folders();    // 更新已有 Folder
    contents = rhs.contents;   // 从 rhs 拷贝消息内容
    folders = rhs.folders;     // 从 rhs 拷贝 Folder 指针
    add_to_Folders(rhs);       // 将本 Message 添加到那些 Folder 中
    return *this;
}
```

如果左侧和右侧运算对象是相同的 `Message`，则它们具有相同的地址。如果我们在 `add_to_Folders` 之后调用 `remove_from_Folders`，就会将此 `Message` 从它所在的所有 `Folder` 中删除。

### Message 的 swap 函数

标准库中定义了 `string` 和 `set` 的 `swap` 版本（参见 9.2.5 节，第 303 页）。因此，如果为我们的 `Message` 类定义它自己的 `swap` 版本，它将从中受益。通过定义一个 `Message` 特定版本的 `swap`，我们可以避免对 `contents` 和 `folders` 成员进行不必要的拷贝。

但是，我们的 `swap` 函数必须管理指向被交换 `Message` 的 `Folder` 指针。在调用 `swap(m1, m2)` 之后，原来指向 `m1` 的 `Folder` 现在必须指向 `m2`，反之亦然。

我们通过两遍扫描 `folders` 中每个成员来正确处理 `Folder` 指针。第一遍扫描将 `Message` 从它们所在的 `Folder` 中删除。接下来我们调用 `swap` 来交换数据成员。最后

对 folders 进行第二遍扫描来添加交换过的 Message:

```
524 void swap(Message &lhs, Message &rhs)
    {
        using std::swap; // 在本例中严格来说并不需要, 但这是一个好习惯
        // 将每个消息的指针从它 (原来) 所在 Folder 中删除
        for (auto f: lhs.folders)
            f->remMsg(&lhs);
        for (auto f: rhs.folders)
            f->remMsg(&rhs);
        // 交换 contents 和 Folder 指针 set
        swap(lhs.folders, rhs.folders); // 使用 swap(set&, set&)
        swap(lhs.contents, rhs.contents); // swap(string&, string&)
        // 将每个 Message 的指针添加到它的 (新) Folder 中
        for (auto f: lhs.folders)
            f->addMsg(&lhs);
        for (auto f: rhs.folders)
            f->addMsg(&rhs);
    }
```

### 13.4 节练习

**练习 13.33:** 为什么 Message 的成员 save 和 remove 的参数是一个 Folder&? 为什么我们不将参数定义为 Folder 或是 const Folder&?

**练习 13.34:** 编写本节所描述的 Message。

**练习 13.35:** 如果 Message 使用合成的拷贝控制成员, 将会发生什么?

**练习 13.36:** 设计并实现对应的 Folder 类。此类应该保存一个指向 Folder 中包含的 Message 的 set。

**练习 13.37:** 为 Message 类添加成员, 实现向 folders 添加或删除一个给定的 Folder\*。这两个成员类似 Folder 类的 addMsg 和 remMsg 操作。

**练习 13.38:** 我们并未使用拷贝和交换方式来设计 Message 的赋值运算符。你认为其原因是什么?



## 13.5 动态内存管理类

某些类需要在运行时分配可变大小的内存空间。这种类通常可以 (并且如果它们确实可以的话, 一般应该) 使用标准库容器来保存它们的数据。例如, 我们的 StrBlob 类使用一个 vector 来管理其元素的底层内存。

但是, 这一策略并不是对每个类都适用; 某些类需要自己进行内存分配。这些类一般来说必须定义自己的拷贝控制成员来管理所分配的内存。

525 例如, 我们将实现标准库 vector 类的一个简化版本。我们所做的一个简化是不使用模板, 我们的类只用于 string。因此, 它被命名为 StrVec。

### StrVec 类的设计

回忆一下, vector 类将其元素保存在连续内存中。为了获得可接受的性能, vector